

초급 백엔드 스터디

3주차 – JPA, DB 설계

지난 주에는...

- 빈 = 공동으로 사용할 **하나의 객체**
- 스프링 컨테이너 = 빈을 저장하는 **공용 공간**
- 의존성 주입 = 필요한 객체를 **전달 받아서 사용하는 것**

- 컨테이너에 빈을 저장하는 방법 : 설정 파일 / **컴포넌트 스캔**
- 컨테이너에서 빈을 주입받는 방법 : **생성자 주입** / 필드 주입

지난 주에는...



이번 주에는...

- 어플리케이션이 사용할 **DB 설계**
- **JPA**를 이용한 **DB 구현**

스프링 Layered Architecture



DB 설계

문제 상황은 크게 **개체**(엔티티)와 그 사이의 **관계**로 나타낼 수 있다.

- **개체**(Entity) : 문제 상황을 구성하는 요소
- **관계**(Relationship) : 개체와 개체 사이의 관계

DB 설계

이렇게 문제 상황을 **개체**와 **관계**로 표현하는 방법을 **ER Model** (Entity-Relationship Model) 이라고 한다.

이때 ER Model을 다이어그램으로 표현한 것을 **ERD** 라고 한다.

DB 설계

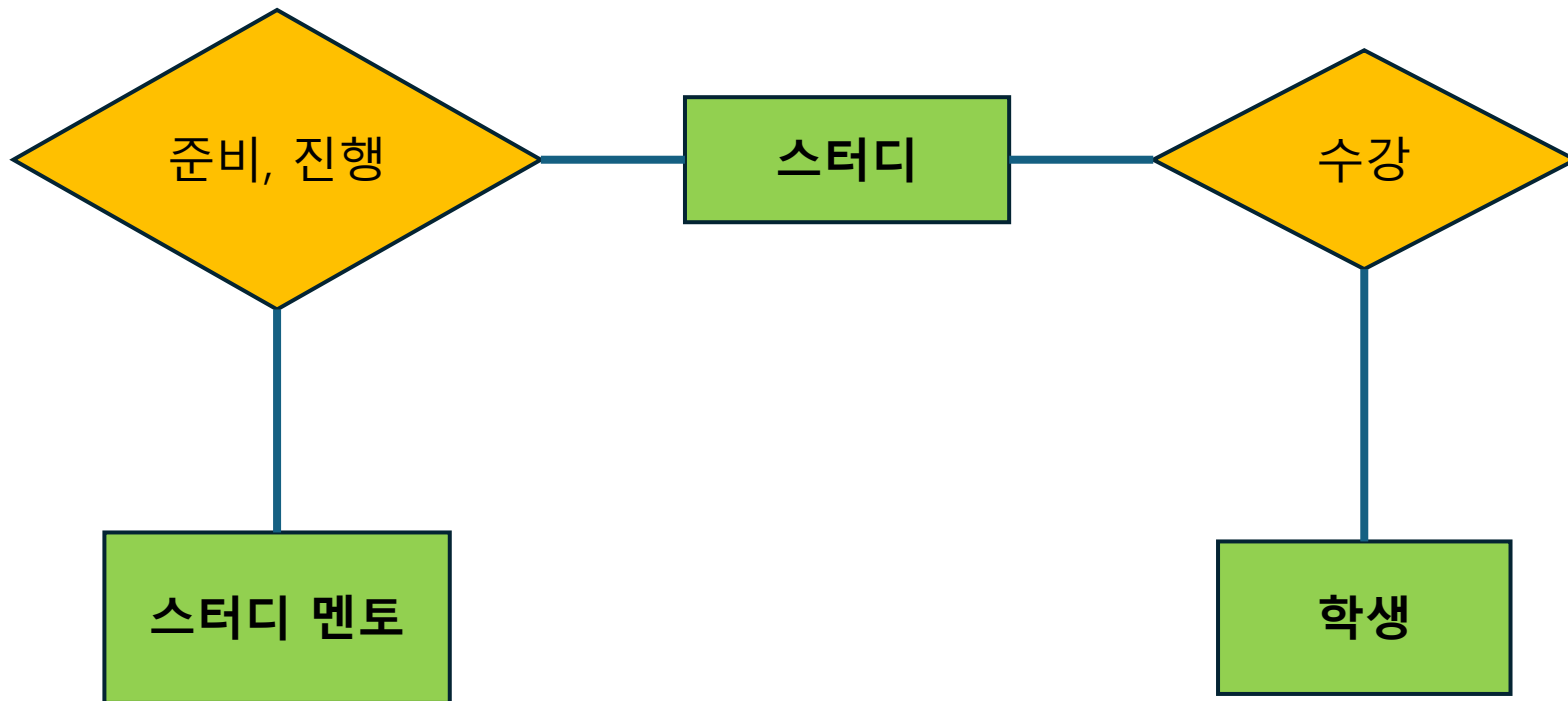
개체와 관계는 세부적인 특징인 **속성**(Attribute)을 가질 수 있다.

‘사람’ 개체는 ‘이름’, ‘나이’ 속성을 가질 수 있고,
‘친구’ 관계는 시작된 ‘연도’ 속성을 가질 수 있다.

이때 하나의 개체를 식별할 수 있는 속성을 **PK**(Primary Key) 라고 한다.

DB 설계

- GDSC 스터디 ERD 예시



DB 설계

ER Model은 다음과 같이 DB로 구현할 수 있다.

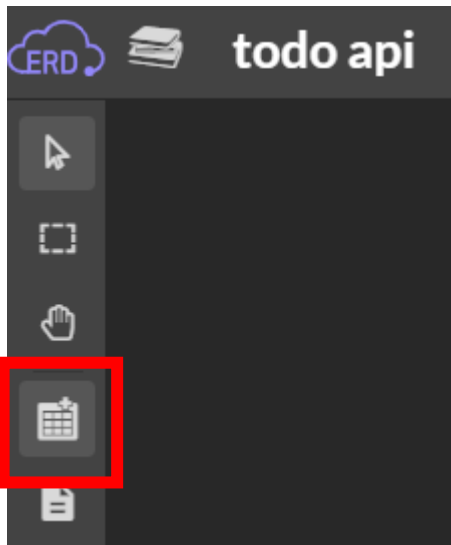
- 개체 → 테이블
- 관계 → 테이블 or 외래키
- 속성 → 테이블 컬럼

DB 설계

- ERD로 DB를 설계해보자.
- ERD를 그리는 도구로 **ERD Cloud**를 사용한다.
(ERD Cloud 말고 자신이 편한 도구를 사용해도 괜찮습니다.)

DB 설계

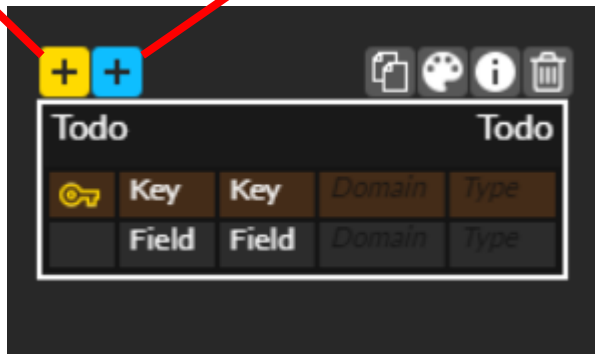
- 엔티티(테이블) 추가



엔티티 생성

PK 추가

컬럼 추가



할 일					Todo				
🔑	할 일 ID	todo_id	Domain	bigint					
	할 일 내용	todo_content	Domain	varchar(200)					
	할 일 완료 여부	todo_is_check	Domain	tinyint(1)					

유저					User				
🔑	유저 ID	user_id	Domain	bigint					
	유저 로그인 아이디	user_login_id	Domain	varchar(20)					

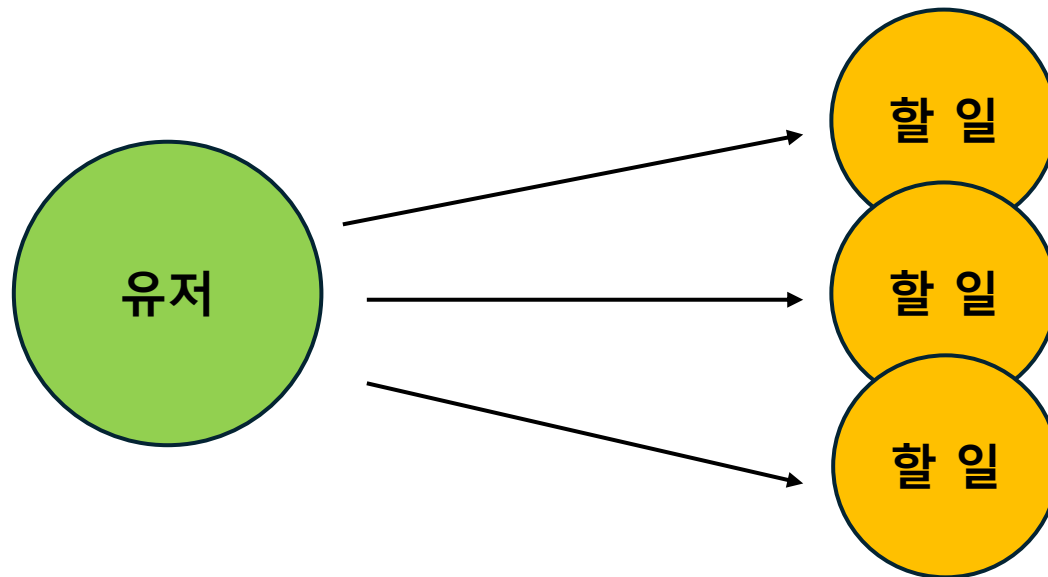
DB 설계

개체와 개체는 사이의 관계는 아래와 같은 종류가 있다.

- 다대일 (N : 1)
- 일대다 (1 : N)
- 일대일 (1 : 1)
- 다대다 (N : M)

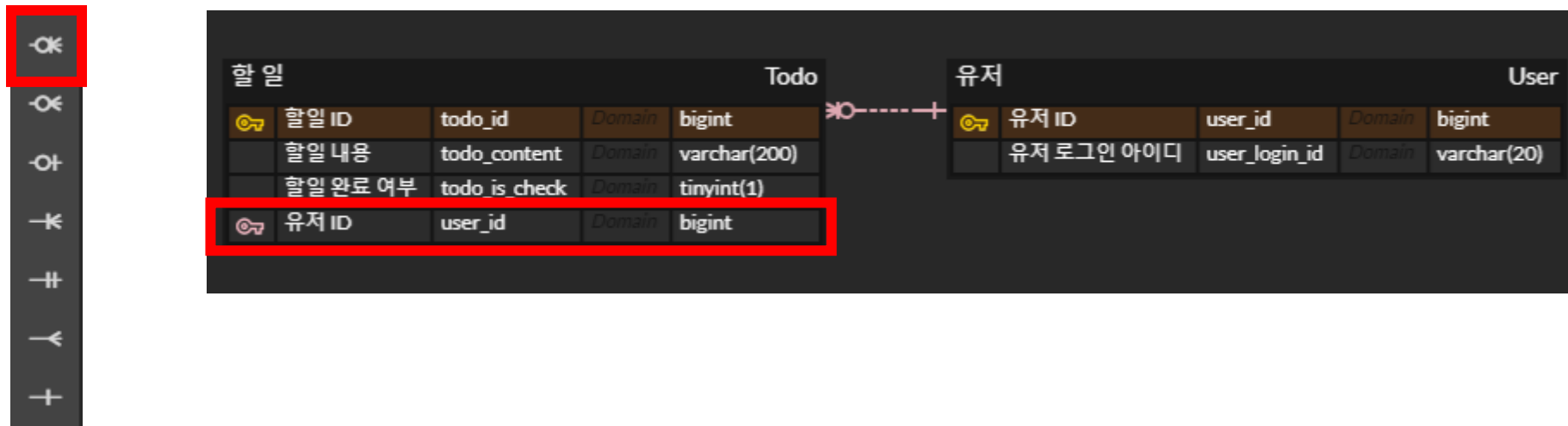
DB 설계

- 1명의 유저는 여러 개의 할 일을 생성할 수 있다.
- 1개의 할 일은 1명의 유저에 대한 할 일이다.
- **유저 - 할 일 관계는 1 : N 관계이다.**



DB 설계

- 1 : N 관계는 외래키로 구현한다. (N : 1 관계는 뒤집어서 동일하게 구현하자)



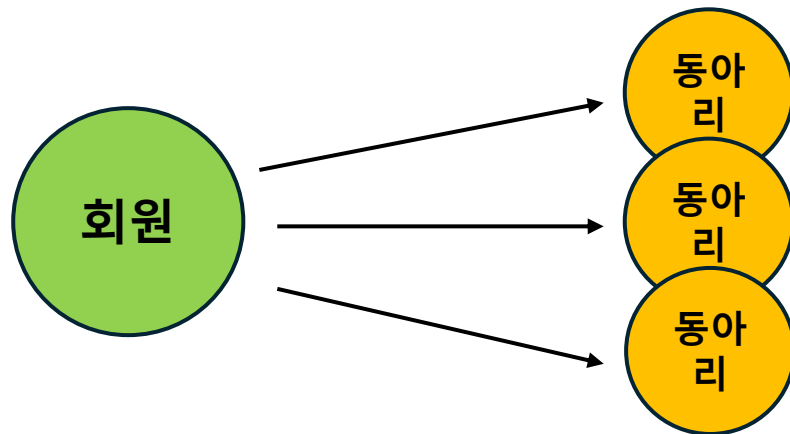
DB 설계

관계를 설정할 때는 보통 **비-식별 관계**를 선택한다.

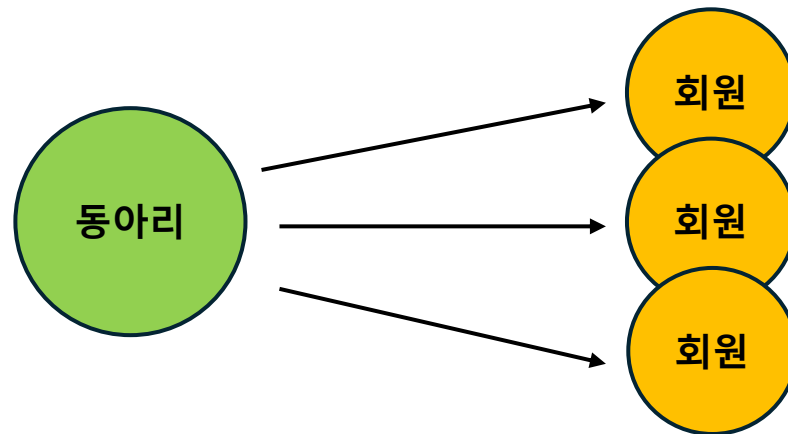
- 식별 관계 : 관계 대상의 PK를 자신의 PK로도 사용하는 것
- 비-식별 관계 : 관계 대상의 PK를 자신의 FK로만 사용하는 것

DB 설계 - N : M

- 회원은 여러 동아리에 소속될 수 있다. (1 : N)
- 동아리는 여러 회원을 가질 수 있다. (1 : M)
- 회원 - 동아리 관계는 **N : M** 이다.



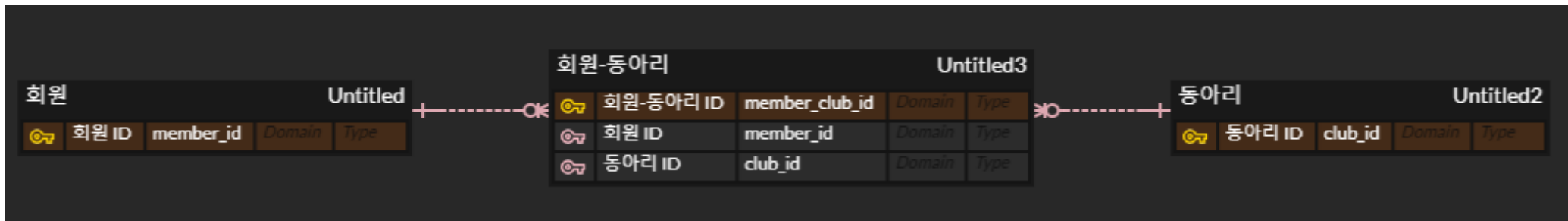
1 : N



1 : M

DB 설계 - N : M

- N : M 관계는 테이블로 구현한다.
- (회원PK - 동아리PK) 쌍을 저장하는 테이블을 만들면 된다.



1 : N

M : 1

DB 설계

관계는 꼭 서로 다른 엔티티끼리 맺지 않아도 된다.

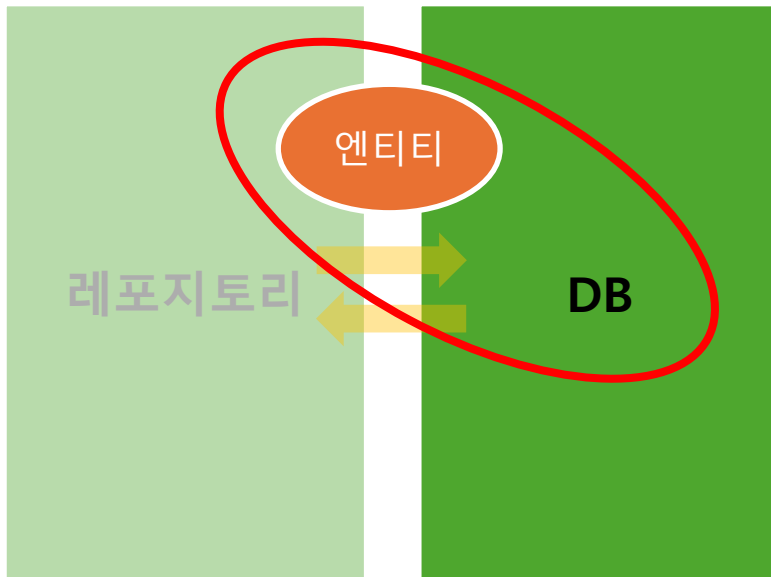
- **유저 – 유저** 사이에 **‘친구’** 관계를 맺는 경우,
같은 종류의 유저 엔티티끼리 관계를 맺을 수 있다.

JPA

- ERD로 설계한 DB를 실제로 구현해보자.

JPA

- Java Persistence API
- 데이터베이스에서 읽어온 데이터를 자바 객체로 매핑하는
자바의 표준 기술 (ORM)



JPA

- **엔티티** (Entity)는 자바와 데이터베이스가 소통하는 단위
- 테이블의 데이터 하나(레코드)는 **엔티티 객체 하나로 매핑**된다.



JPA

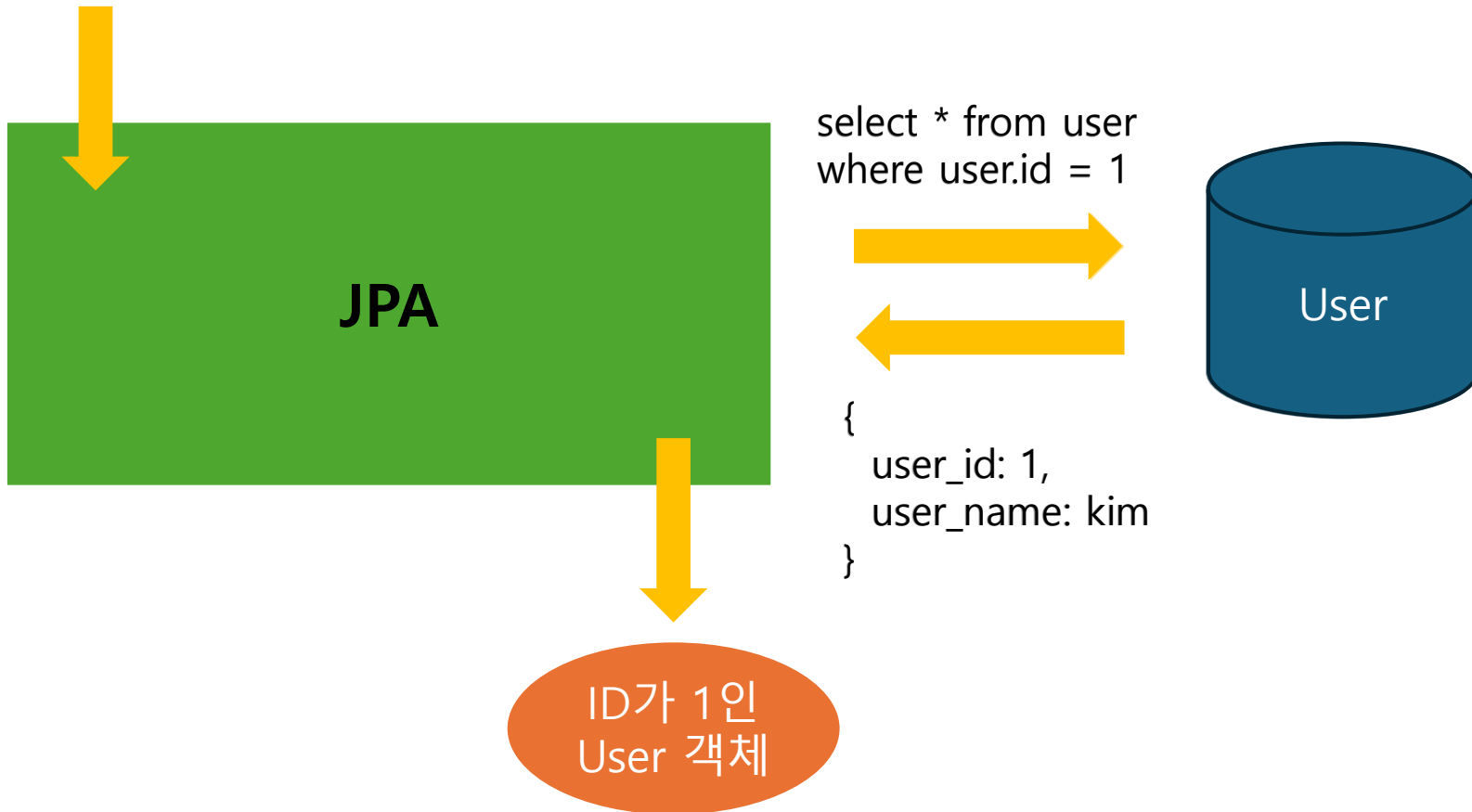
- 엔티티 클래스를 정의하면, JPA가 엔티티 클래스 정의를 보고 **테이블을 생성하는 SQL**을 알아서 작성하고 실행한다.

(+ 다음 주에 다루는 내용이지만, CRUD SQL도 알아서 작성하고 실행해준다)

→ JPA를 사용함으로써 SQL을 작성하는 시간을 줄일 수 있다.

JPA

id가 1인 유저 데이터 조회



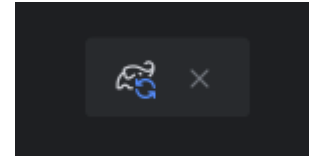
JPA

- 이제 **데이터베이스**를 준비하고, JPA로 테이블을 만들어보자.
- DB는 간단하게 사용할 수 있는 **H2 데이터베이스**를 사용한다.

JPA - 의존성 추가

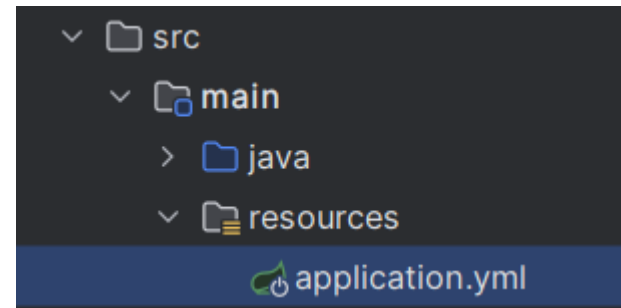
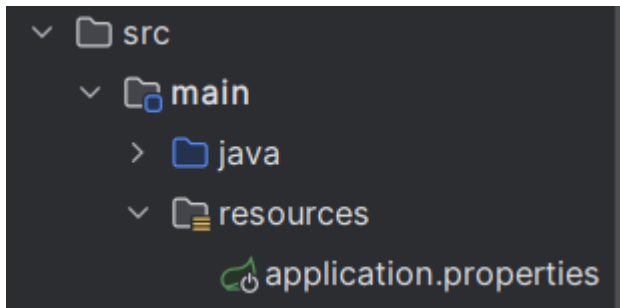
- **build.gradle**에 **JPA**와 **H2 데이터베이스** 의존성을 추가한다.
- 의존성 정보가 바뀌면 반드시 gradle을 다시 로드하자

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    compileOnly 'org.projectlombok:lombok'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
  
    // H2 DB  
    implementation 'com.h2database:h2'  
}
```



JPA – DB 연결 정보 추가

- resources – **application.properties** 파일에 DB 접속 정보를 작성한다.
- 편의를 위해 이 파일의 확장자를 **yml**로 바꾼다.



JPA – DB 연결 정보 추가

- yml 파일은 아래와 같이 콜론(:)을 사용하여 구분한다.

```
1  ∨ spring:
2  ∨   application:
3     name: todo-api
4
```

JPA – 설정 파일 작성

- 다음과 같이 데이터베이스 정보를 작성한다.

```
spring:
  application:
    name: todo-api

  datasource:
    url: jdbc:h2:mem:todo;MODE=MYSQL

  h2:
    console:
      enabled: true

  jpa:
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.MySQL8Dialect
```

JPA – 설정 파일 작성

- 스프링 부트는 기본적으로 H2 in-memory DB를 사용한다.
- DB에 저장된 데이터를 볼 수 있도록 **관리자 콘솔**을 활성화하고, 관리자 콘솔에 접속할 url을 명시한다.

```
spring:
  application:
    name: todo-api

  datasource:
    url: jdbc:h2:mem:todo;MODE=MYSQL

  h2:
    console:
      enabled: true
```

관리자 콘솔에 접속할 url
설정하지 않으면 매번 랜덤 url 생성

관리자 콘솔 활성화.
기본값은 false

JPA - 설정 파일 작성

- JPA 관련 설정

```
jpa:  
  show-sql: true  
  properties:  
    hibernate:  
      format_sql: true  
      dialect: org.hibernate.dialect.MySQL8Dialect
```



JPA가 생성한 SQL을 표시하고,
보기 좋게 들여쓰기 적용



SQL을 생성할 때, MySQL 8 문법 사용

JPA – 관리자 콘솔 접속

- 어플리케이션을 실행하고, **localhost:8080/h2-console** 에 접속
- 아래 그림과 같이 설정한 뒤, Connect 클릭

English Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded)

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:todo

User Name: sa

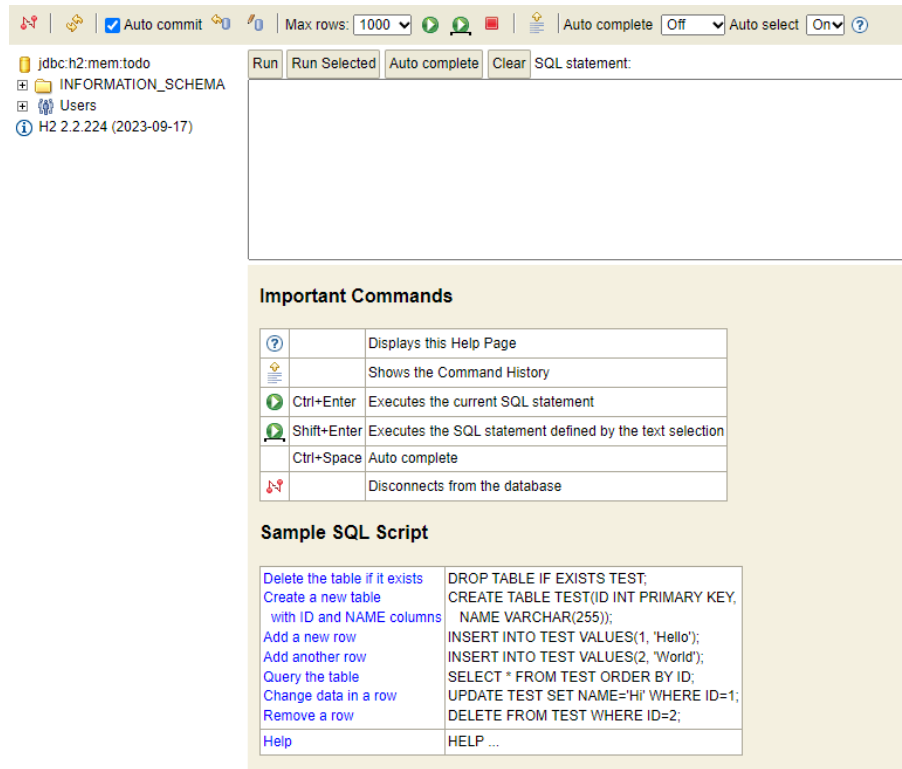
Password:

Connect Test Connection

JDBC URL에 application.yml 파일에
작성한 URL 입력

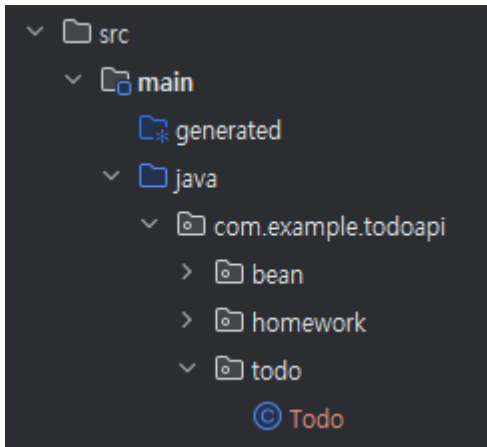
JPA – 관리자 콘솔 접속

- 그림과 같이 나오면 성공



엔티티 클래스

- **todo** 패키지를 만들고, 안에 **Todo** 클래스 생성
- 엔티티 클래스는 **테이블**을, 클래스 필드는 **컬럼**을 나타낸다.



```
public class Todo { 0개의 사용위치

    private Long id; 0개의 사용위치

    private String content; 0개의 사용위치

    private boolean isChecked; 0개의 사용위치
}
```

엔티티 클래스

- **@Entity** 어노테이션으로 이 클래스가 엔티티라는 것을 명시
- **@Id** 어노테이션으로 PK 필드에 이 필드가 PK라는 것을 명시

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity 0개의 사용위치
public class Todo {

    @Id
    private Long id;

    private String content; 0개의 사용위치

    private boolean isChecked; 0개의 사용위치
}
```

엔티티 클래스

- id 값은 보통 데이터를 생성할 때마다 자동으로 1씩 늘어난다.
- **@GeneratedValue**를 사용하면 id 값을 자동으로 생성한다.
- 이때 strategy는 IDENTITY로 설정한다. (키 값 결정을 DB에 위임)

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

엔티티 클래스

- 어플리케이션을 실행하면
웹에서 테이블 생성 SQL이 실행된 것을 확인할 수 있다.
- 콘솔에서도 Todo 테이블이 생성된 것을 볼 수 있다.

Hibernate:

```
create table todo (  
  todo_is_check tinyint(1),  
  todo_id bigint not null auto_increment,  
  user_id bigint,  
  todo_content varchar(200),  
  primary key (todo_id)  
) engine=InnoDB
```

jdbc:h2:mem:todo
+ TODO
+ INFORMATION_SCHEMA
+ Users
i H2 2.2.224 (2023-09-17)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM TODO|

SELECT * FROM TODO;

IS_CHECKED ID CONTENT

(no rows, 2 ms)

Edit

엔티티 클래스

- ERD에서 설계했던 Column 이름과 타입을 맞추기 위해, 필드에 **@Column** 으로 이름과 타입을 명시한다.

```
@Entity 0개의 사용위치
public class Todo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "todo_id")
    private Long id;

    @Column(name = "todo_content", columnDefinition = "varchar(200)") 0개의 사용위치
    private String content;

    @Column(name = "todo_is_check", columnDefinition = "tinyint(1)") 0개의 사용위치
    private boolean isChecked;
}
```

SELECT * FROM TODO;

TODO_IS_CHECK	TODO_ID	TODO_CONTENT
---------------	---------	--------------

(no rows, 2 ms)

Edit

엔티티 연관관계

- 외래키 컬럼을 나타낼 때는 Long 타입의 외래키 필드 대신, 해당 엔티티 타입의 **엔티티 객체**를 필드로 가지도록 설계한다.

```
@ManyToOne(fetch = FetchType.LAZY) 0개의 사용위치  
@JoinColumn(name = "user_id")  
private User user;
```

SELECT * FROM TODO;

TODO_IS_CHECK	TODO_ID	USER_ID	TODO_CONTENT
---------------	---------	---------	--------------

(no rows, 2 ms)

Edit

외래키를 직접 저장한다면?

- 만약 외래키를 직접 저장한다면 연관된 데이터가 필요할 때, 외래키로 데이터를 조회하는 코드를 **직접 작성**해야 한다.
- 엔티티로 저장하면 테이블을 만들 때 외래키를 만들어주고, 연관된 데이터가 필요할 때, **자동으로 join 쿼리가 실행**되면서 연관된 데이터를 얻는다.

엔티티 연관관계

외래키 필드에는 2가지 어노테이션을 지정해주어야 한다.

- FK 컬럼 정보를 명시하는 어노테이션 (컬럼 이름 등)
@JoinColumn
- 해당 외래키로 생기는 연관관계 종류를 나타내는 어노테이션
@ManyToOne @OneToOne @OneToMany @ManyToMany

엔티티 연관관계

- **@JoinColumn**은 외래키 컬럼 정보를 나타낸다.

```
@ManyToOne(fetch = FetchType.LAZY) 0개의 사용위치  
@JoinColumn(name = "user_id")  
private User user;
```



FK 컬럼 이름 지정

엔티티 연관관계

이번 스터디에서는 다음 2가지 필요에 따라 사용한다.

- **@ManyToOne** : N : 1 관계
- **@OneToOne** : 1 : 1 관계 (필요시 사용)

엔티티 연관관계

- **@ManyToMany** 는 $N : M$ 관계를 나타낸다.
 $N : M$ 관계는 외래키대신 **테이블로 구현**하므로 사용하지 않는다.
- **@OneToMany** 는 $1 : N$ 관계를 나타낸다.
1에 해당하는 엔티티에 N 에 대한 연관관계를 명시하는 **양방향 매핑**에 사용된다. (이번 스터디에서는 **단방향 매핑**만 사용한다.)

엔티티 연관관계

- 연관관계 종류를 나타내는 어노테이션에는 **fetch** 속성이 있다.
- 이 속성으로 연결된 엔티티를 **언제 가져올지 명시**할 수 있다.

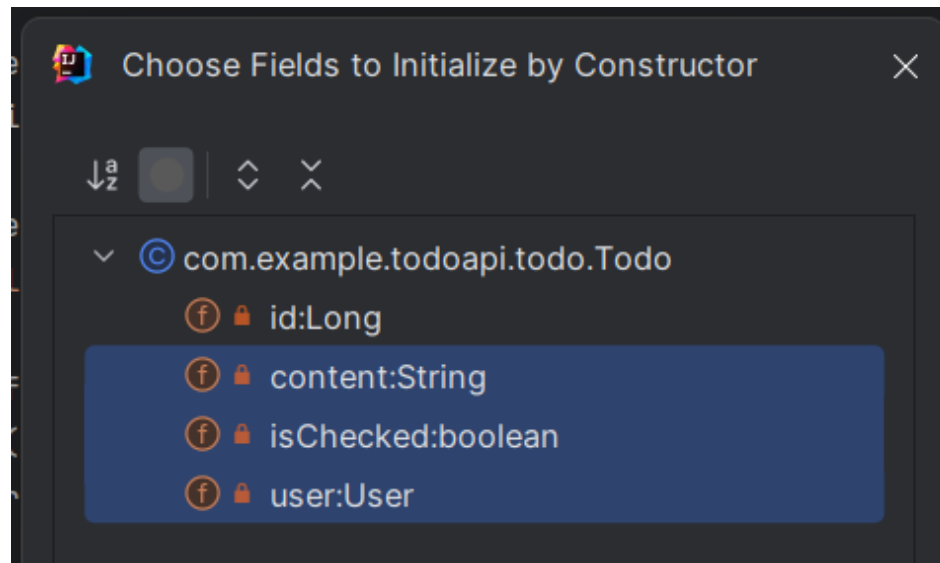
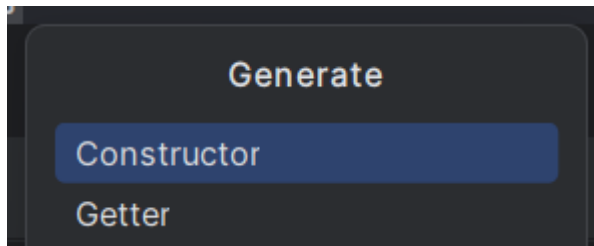
```
@ManyToOne(fetch = FetchType.LAZY) 0개의 사용위치  
@JoinColumn(name = "user_id")  
private User user;
```

엔티티 연관관계

- fetch type에는 EAGER, LAZY 2가지가 있는데, **LAZY**를 사용하자.
- EAGER : **즉시 로딩**, Todo 객체 정보를 가져올 때 연결된 User 객체의 모든 정보를 **함께 한번에 가져온다**.
- LAZY : **지연 로딩**, Todo 객체 정보를 가져올 때 연결된 User 객체의 정보는 **필요할 때 가져온다**.

엔티티 생성자

- 엔티티 객체를 생성할 생성자를 만든다.
- alt+insert키를 눌러 id를 제외한 필드에 대한 생성자를 추가한다.
(id 값은 자동으로 생성된다.)



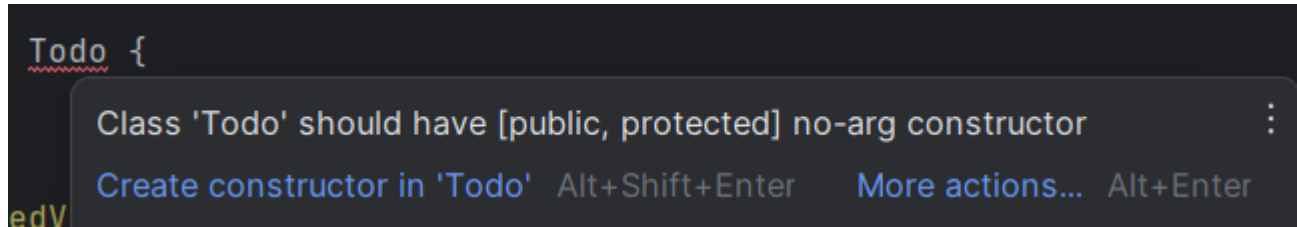
엔티티 생성자

- 아래와 같이 생성자가 만들어진다.
필요에 따라 **@Builder** 와 같은 어노테이션을 사용하기도 한다.
(@Builder는 한번 찾아서 공부해보세요!)

```
public Todo(String content, boolean isChecked, User user) {  
    this.content = content;  
    this.isChecked = isChecked;  
    this.user = user;  
}
```


엔티티 생성자

- JPA는 엔티티 객체를 다룰 때 public 또는 protected의 **인자 없는 생성자가 필요하다**.



엔티티 생성자

- **@NoArgsConstructor**를 사용하여 인자 없는 생성자를 만든다.
- 이때 access 속성을 통해 접근 제한자를 protected로 설정한다.
- 추가로 엔티티 객체에 **@Getter**를 추가해 모든 필드에 getter를 만든다.

```
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Todo {
```

정리

- ERD를 그려서 DB를 설계할 수 있다.
- JPA는 DB와 자바 객체를 매핑하는 도구이다.
이때 DB와 매핑되는 자바 객체를 **엔티티**라고 한다.
- JPA를 사용해서 엔티티 클래스로 DB를 정의할 수 있다.

정리

- 엔티티 클래스에는 **@Entity**, **@Id** 어노테이션이 필요하다.
- Id값이 자동 생성될 땐 **@GeneratedValue** 를 사용한다.
- 일반 컬럼에는 **@Column** 으로 컬럼 명, 컬럼 타입 등을 지정한다.

정리

- 외래키를 나타낼 땐 엔티티 객체를 필드로 넣고 **@JoinColumn, @ManyToOne** 을 사용한다.
- **@ManyToOne** 에서 **fetch** 속성은 **LAZY**로 지정한다.

정리

- 엔티티 생성자는 보통 id 필드를 제외하고 만든다.
- JPA가 엔티티를 사용하려면 **인자 없는 생성자가 필요**하며,
@NoArgsConstructor 어노테이션으로 간편하게 만들 수 있다.

프로젝트 - 과제

- 프로젝트 기능 명세를 보고 직접 **ERD**를 그려보자
- ERD를 참고하여 **엔티티 클래스를 작성**하자
- 어플리케이션을 실행하고, H2 데이터베이스 관리자 콘솔에서 **테이블이 생성된 것을 확인**하자

프로젝트 명세

Todo mate API 서버 클론 코딩

<주요 기능>

- 유저 회원가입, 로그인
- 로그인한 유저의 할 일 생성 / 조회 / 수정 / 삭제
- 로그인한 유저의 할 일 체크 / 체크 해제
- 다른 유저에게 친구 요청 / 요청 수락 / 친구 조회 / 친구 삭제
- 친구 유저의 할 일 조회

프로젝트 명세

- User 엔티티는 자유롭게 설계하자.
- 친구는 유저와 유저 사이의 관계이다.
이 관계는 **1 : 1 / 1 : N / N : M** 중 어떤 관계에 속할까?
- N : M 관계는 중간 테이블로 나누는 것을 기억하자!